

Database systems 1

dr hab. inż. Przemysław Korytkowski, prof. ZUT

Lecture 9 – XML and JSON

1

1

Types of data

Structured

- Data is collected and inserted into carefully designed structured databases.

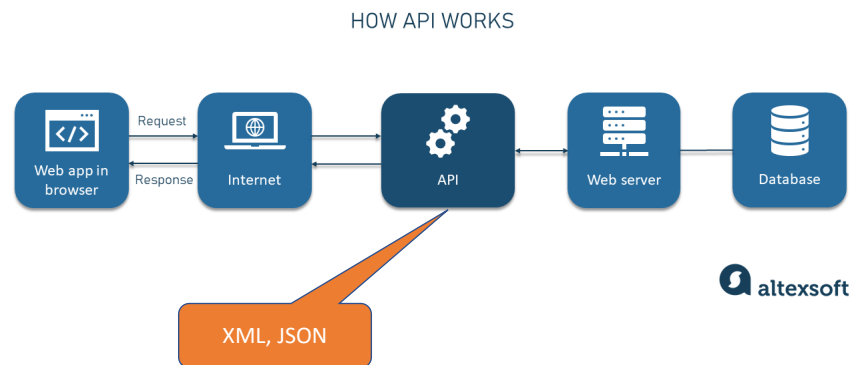
Semistructured (self-describing)

- There is no requirement for a predefined schema to which the data objects must conform, although it is possible to define a schema if necessary.
- Some attributes may be shared among the various entities, but other attributes may exist only in a few entities. Moreover, additional attributes can be introduced in some of the newer data items at any time, and there is no predefined schema.
- The information schema is mixed in with the data values, since each data object can have different attributes that are not known in advance.

2

2

Application Programming Interfaces



3

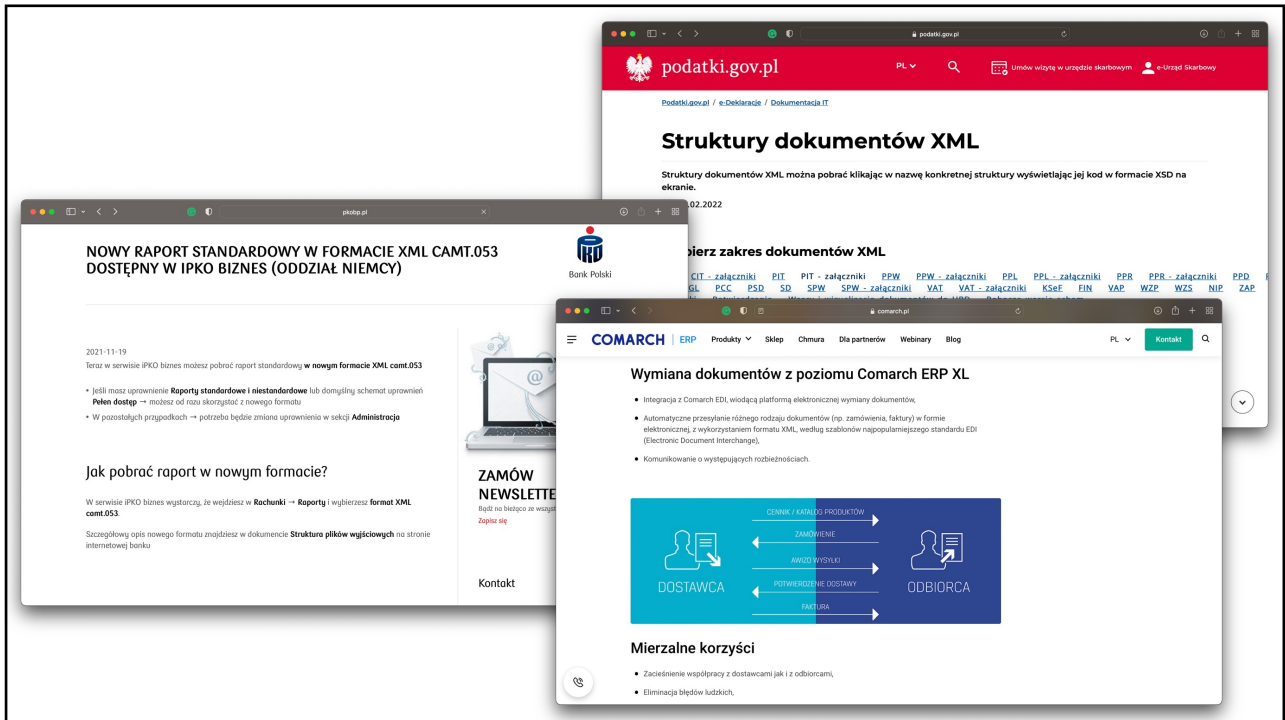
3

XML

- Developed in 1996 under W3C (World Wide Web Consortium)
- Current version: Extensible Markup Language (XML) 1.1 (Second Edition), W3C Recommendation 16 August 2006, edited in place 29 September 2006
- <https://www.w3.org/TR/xml11>
- Technologies:
 - XML - eXtensible Markup Language
 - DTD - Document Type Definition
 - XSD - XML Schema Definition
 - XPath - XML Path Language
 - XQuery – XML Query
 - XSL - Extensible Stylesheet Language - *not covered*
 - XSLT - XSL Transformations - *not covered*

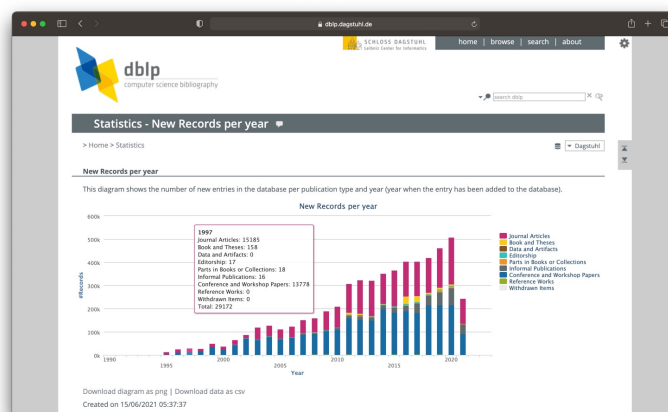
4

4



5

dblp computer science bibliography (dblp.org)



6

6

XML

The diagram illustrates the structure of an XML document. It shows a sequence of elements: an XML declaration, a `<dblpperson>` element, an `<article>` element, and their closing tags. Callouts identify specific parts: 'XML declaration' points to the first line; 'Attribute' points to the `name` attribute of `<dblpperson>`; 'Element' points to the `<person>` element; 'Tag' points to the opening and closing tags of `<person>`; and 'Complex element' points to the `<article>` element, which contains multiple sub-elements like `<author>`, `<title>`, `<pages>`, etc.

```

<?xml version="1.0" encoding="UTF-8"?>
<dblpperson name="E. F. Codd" pid="c/EFCodd" n="51">
  <person key="homepages/c/EFCodd" mdate="2020-05-20">
    <author pid="c/EFCodd">E. F. Codd</author>
    <author pid="c/EFCodd">Edgar F. Codd</author>
    <note type="award" label="1981">Turing Award</note>
    <note type="award" label="1996">Computer Pioneer Award</note>
    <url>https://dl.acm.org/profile/81100425534</url>
  </person>
</dblpperson>

<article key="journals/cacm/Codd70" mdate="2018-11-14">
  <author pid="c/EFCodd">E. F. Codd</author>
  <title>A Relational Model of Data for Large Shared Data Banks.</title>
  <pages>377-387</pages>
  <year>1970</year>
  <volume>13</volume>
  <journal>Commun. ACM</journal>
  <number>6</number>
  <url>db/journals/cacm/cacm13.html#Codd70</url>
</article>

```

7

źródło: dblp computer science bibliography (<https://dblp.org>)

7

XML – tree structure

The diagram shows the hierarchical tree structure of the XML document. The root node is `dblp`. It has three main branches: `article`, `book`, and `www`. The `article` branch further divides into `author`, `title`, and `journal`. The `title` node has a sub-branch `sup`. The `book` and `www` branches lead to `...` nodes, indicating further content.

```

<?xml version="1.0" encoding="UTF-8"?>
<dblpperson name="E. F. Codd" pid="c/EFCodd" n="51">
  <person key="homepages/c/EFCodd" mdate="2020-05-20">
    <author pid="c/EFCodd">E. F. Codd</author>
    <author pid="c/EFCodd">Edgar F. Codd</author>
    <note type="award" label="1981">Turing Award</note>
    <note type="award" label="1996">Computer Pioneer Award</note>
    <url>https://dl.acm.org/profile/81100425534</url>
  </person>
  <article key="journals/cacm/Codd70" mdate="2018-11-14">
    <author pid="c/EFCodd">E. F. Codd</author>
    <title>A Relational Model of Data for Large Shared Data Banks.</title>
    <pages>377-387</pages>
    <year>1970</year>
    <volume>13</volume>
    <journal>Commun. ACM</journal>
    <number>6</number>
    <url>db/journals/cacm/cacm13.html#Codd70</url>
  </article>
</dblpperson>

```

8

źródło: dblp computer science bibliography (<https://dblp.org>)

8

XML Data Components

- Elements `<article key="journals/cacm/Codd70" mdate="2018-11-14">`
`<author pid="c/EFcodd">E. F. Codd</author>...`
`</article>`
 - Hierarchical structure with open tag – close tag pairs
 - May include nested elements
 - May include attributes within the element's open-tag
 - Multiple elements may have same name
 - Order matters
 - Names cannot start with the letters xml (or XML, or Xml, etc)
 - Names can contain letters, digits, hyphens, underscores, and periods.
 - Names cannot contain spaces.
- Attributes `key="journals/cacm/Codd70"`
 - Named values – not hierarchical
 - Only one attribute with a given name per element
 - Order does NOT matter

9

9

Well-formed XML

Any legal XML document is always parsable by an XML parser, without knowledge of tag meaning

- The start – *preamble* – tells XML about the char. encoding
`<?xml version="1.0" encoding="utf-8"?>`
- There's a single root element
- All open-tags have matching close-tags (unlike many HTML documents!), or a special:
`<tag/>` shortcut for empty tags (equivalent to `<tag></tag>`)
- Attributes only appear once in an element
- XML is case-sensitive

10

10

XML as a Data Model

XML “information set” includes 7 types of nodes:

- Document (root)
- **Element**
- **Attribute**
- Processing instruction
- **Text (content)**
- Namespace
- Comment

XML data model includes this, plus typing info, plus order info and a few other things

11

11

XML Easily Encodes Relations

```
<student-course-grade>
  <tuple>
    <sid>1</sid>
    <cid>570103</cid>
    <grade>B</grade>
  </tuple>
  <tuple>
    <sid>23</sid>
    <cid>550103</cid>
    <grade>A</grade>
  </tuple>
</student-course-grade>
```

student-course-grade

sid	cid	grade
1	570103	B
23	550103	A

```
<student-course-grade>
  <tuple sid="1" cid="570103" grade="B"/>
  <tuple sid="23" cid="550103" grade="A"/>
</student-course-grade>
```

12

12

Namespaces

- *Namespaces* allow us to specify a context for different tags
- Two parts:
 - Binding of namespace to URI
 - Qualified names

```
<root xmlns="http://www.first.com/" xmlns:second="http://www.second.com/">
  <third:aaa xmlns:third="http://www.page.com/third">
```

Defines "third"
for qualified
names

Default namespace for
non-qualified names

```
<tag>is in the default namespace (www.first.com/)</tag>
<third:tag>is in third</third:tag>
<second:tag>is a different tag in second</second:tag>
</third:aaa>
</root>
```

Defines "second"
for qualified names

13

13

XML isn't Enough on Its Own

It's too unconstrained for many cases!

- How will we know when we're getting garbage?
- How will we know what to query for?
- How will we understand what we received?

We also need:

- An idea of (at least part of) the structure
- Some knowledge of how to interpret the tags...

Solutions:

- DTD – Document Type Definition
- XSD – XML Schema

14

14

Document Type Definitions (DTDs)

The DTD is an EBNF grammar (context-free grammar) defining XML structure

- The XML document specifies an associated DTD, plus the root element of the document
- DTD specifies children of the root (and so on)

DTD also defines special attribute types:

- **IDs** – special attributes that are analogous to keys for elements
- **IDREFs** – references to IDs
- **IDREFS** – a list of **IDREFs**, space-delimited
- All other attributes are essentially treated as **strings**

15

15

DTD example

Example DTD:

```
<!ELEMENT dblp
(article|inproceedings|proceedings|book|incollection|phdthesis|mastersthesis|www|person|data)*>
  <!ATTLIST dblp mdate CDATA #IMPLIED >
  <!ENTITY % field "author|editor|title|booktitle|pages|year|address|journal|volume|number|month|
    url|ee|cdrom|cite|publisher|note|crossref|isbn|series|school|chapter|publnr">
  <!ELEMENT article (%field;)*>
  <!ATTLIST article
    key CDATA #REQUIRED
    mdate CDATA #IMPLIED
    publname CDATA #IMPLIED
    ...
  > ...
```

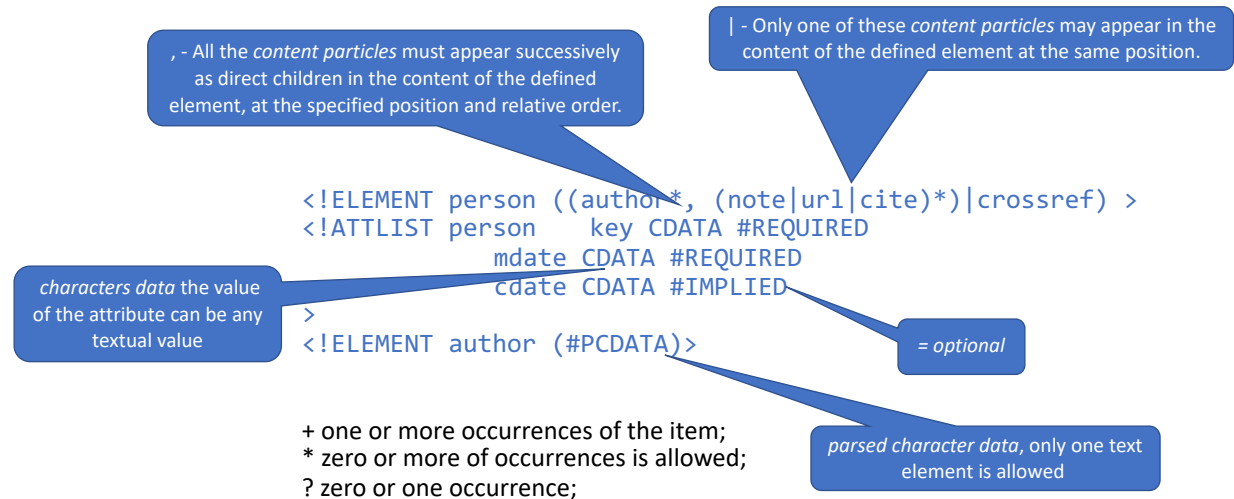
Example use of DTD in XML file:

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE dblp SYSTEM "dblp.dtd">
<dblp>...
```

źródło: dblp computer science bibliography (<https://dblp.org>) ¹⁶

16

DTD example



17

17

Limitations of DTDs

DTDs capture grammatical structure, but have some drawbacks:

- Don't capture database datatypes' domains.
- DTDs basically have one data type: the text string.
- DTDs are not object oriented. There is no inheritance in DTDs.
- DTDs are composed of non-XML syntax – inconvenient to build tools for them
- There can only be a single DTD per document. All aspects of an XML document must be within a single DTD.
- The ability of the internal DTD subset to override the external DTD subset. The settings in the DTD could be overridden by the internally defined elements of a DTD.

18

18

XML Schema (XSD)

Aims to address the shortcomings of DTDs

- XML syntax
- Can define keys using XPaths
- Type subclassing that also includes restrictions on ranges
 - “By extension” (adds new data) and “by restriction” (adds constraints)
- Domains and built-in datatypes

(Note there are other XML schema formats like RELAX NG)

19

19

Basics of XML Schema

Need to use the XML Schema namespace (generally named xsd)

- **simpleTypes** are a way of restricting domains on scalars
 - Can define a **simpleType** based on integer, with values within a particular range
- **complexType** are a way of defining element/attribute structures
 - Basically equivalent to **!ELEMENT**, but more powerful
 - Specify sequence, choice between child elements
 - Specify **minOccurs** and **maxOccurs** (default 1)
- Must associate an **element/attribute** with a **simpleType**, or an **element** with a **complexType**

20

20

XSD – XML Schema Example

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema" elementFormDefault="qualified">
  <xs:element name="dblpsubmission">
    <xs:complexType>
      <xs:choice maxOccurs="unbounded" minOccurs="0">
        <xs:element name="proceedings">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="title" type="xs:string"/>
              <xs:element name="year" type="xs:string"/>
              <xs:element ref="isbn" minOccurs="0" maxOccurs="unbounded"/>
              <xs:element ref="doi" minOccurs="0"/>
            </xs:sequence>
          </xs:complexType>
        </xs:element>
        <xs:sequence>
          <xs:element name="acronym" type="xs:string" minOccurs="0"/>
          <xs:element name="number" type="xs:string" minOccurs="0"/>
          <xs:element name="date" type="xs:string"/>
        </xs:sequence>
      </xs:choice>
    </xs:complexType>
  </xs:element>
```

źródło: dblp computer science bibliography (<https://dblp.org>) ²¹

21

XSD – restrictions

```
<xs:element name="password">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:pattern value="[a-zA-Z0-9]{8}" />
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

Exactly eight characters in a row and those characters must be lowercase or uppercase letters from a to z, or a number from 0 to 9

```
<xs:element name="password">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:minLength value="5" />
      <xs:maxLength value="8" />
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

The value must be minimum five characters and maximum eight characters

22

22

XSD – XML Schema

- 19 primitive data types: anyURI, base64Binary, boolean, date, dateTime, decimal, double, duration, float, hexBinary, gDay, gMonth, gMonthDay, gYear, gYearMonth, NOTATION, QName, string, and time.
- Complex types describe the permitted content of an element, including its element and text children and its attributes.
- Drawbacks:
 - It is too complicated (the spec is several hundred pages in a very technical language).
 - XSD lacks any formal mathematical specification.
 - Restriction of elements works differently from restriction of attributes.
 - XSD cannot require a specific *root element* (so extra information is required to validate even the simplest documents).
 - XSD offers very weak support for unordered content.

23

23

Designing an XML Schema/DTD

Not as formalized as relational data design

- Typically based on an existing underlying design, e.g., relational DBMS or spreadsheet

We generally orient the XML tree around the “central” objects

Big decision: element vs. attribute

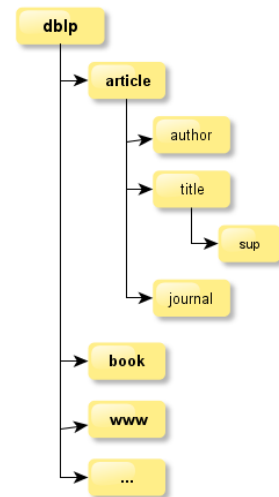
- Element if it has its own properties, or if you *might* have more than one of them
- Attribute if it is a single property – though element is OK here too!

24

24

Document Object Model (DOM) and Simple API for XML (SAX)

- A huge benefit of XML – standard parsers and standard (cross-language) APIs for processing it.
- DOM: an object-oriented representation of the XML parse tree (roughly like the Data Model graph)
 - **DOM objects** have methods like “getFirstChild()”, “getNextSibling”
 - Common way of traversing the tree
 - Can also modify the DOM tree – alter the XML – via insertAfter(), etc.
 - A parser must build a tree representation of the entire document in memory.
- Sometimes we don’t want all the data: SAX
 - **Parser interface** that calls a function each time it parses a processing-instruction, element, etc.
 - Your code can determine what to do, e.g., build a data structure, or discard a particular portion of the data
 - the minimum memory required for a SAX parser is proportional to the maximum depth of the XML file.



25

25

XPath

- A query language for retrieving data from XML document.
- Xpath could also compute values from the content of an XML document.
- XPath uses "path like" syntax to identify and navigate nodes in an XML document
- XPath contains over 200 built-in functions

26

26

XPath - examples

```
<result>
  <hits total="5" computed="5" sent="5" first="0">
    <hit score="12" id="2990687">
      <info>
        <authors>
          <author>Erik Meijer</author>
          <author>Gavin M. Bierman</author>
        </authors>
        <title>A co-relational model of data for large
          shared data banks.</title>
        <venue>Commun. ACM</venue>
        <volume>54</volume>
        <number>4</number>
        <pages>49-58</pages>
        <year>2011</year>
        <type>Journal Articles</type>
        <doi>10.1145/1924421.1924436</doi>
      </info>
      <url>URL#2990687</url>
    </hit>
  </hits>
  ...
</result>
```

```
/result/hits/
```

```
/result/hits/hit[1]/info/
```

```
/result/hits/hit[1]/info/authors/author[2]
```

```
/result/hits/hit[1]/info/venue/text()
```

27

27

XPath - examples

```
<result>
  <hits total="5" computed="5" sent="5" first="0">
    <hit score="12" id="2990687">
      <info>
        <authors>
          <author>Erik Meijer</author>
          <author>Gavin M. Bierman</author>
        </authors>
        ...
      </info>
      <url>URL#2990687</url>
    </hit>
  </hits>
  ...
  <hit score="2" id="1430304">
    <info>
      <authors>
        <author>Robert Codd-Downey</author>
        <author>Erik Meijer</author>
        <author>Katherine Allison</author>
      </authors>
    </info>
  </hit>
  ...
</result>
```

```
/result/hits //info/authors/author
```

```
//info[authors/author = "Erik Meijer"]
```

```
//info[./authors/author.text() = "Erik Meijer"]
```

```
/result/hits/hit/@id
```

```
/result/hits/hit[@id="2990687"]/url/text()
```

28

28

Beyond XPath: XQuery

A strongly-typed, Turing-complete XML manipulation language

- Attempts to do static typechecking against XML Schema
- Based on an object model derived from Schema

Unlike SQL, fully compositional, highly orthogonal:

- Inputs & outputs collections (sequences or bags) of XML nodes
- Anywhere a particular type of object may be used, may use the results of a query of the same type
- Designed mostly by DB and functional language people

Can be used to define queries, views, and (using a subset) schema mappings

29

29

XQuery's Basic Form

- Has an analogous form to SQL's **SELECT..FROM..WHERE..GROUP BY..ORDER BY**
- The model: bind nodes (or node sets) to variables; operate over each legal combination of bindings; produce a set of nodes
- “FLWOR” statement [note case sensitivity!]:
 - for** {iterators that bind variables}
 - let** {collections}
 - where** {conditions}
 - order by** {order-paths}
 - return** {output constructor}
- Mixes XML + XQuery syntax; use {} as “escapes”

30

30

XQuery – example 1

```
<publications> {
  for $publication in doc ("dblp.xml")/result/hits/*/info,
    $year in $publication/year
  where $year = "2015"
  return <paper> { $publication/title } </paper>
} </publications>
```

XPath

```
<publications>
  <paper>A co-relational model of data for large shared data banks.</paper>
  <paper>Fatal Flaws in SQL - Part ONE.</paper>
  <paper>Fatal Flaws in SQL - Part Two.</paper>
  ...
</publications>
```

31

31

XQuery – example 2

```
for $x in doc ("dblp.xml") /result/hits/*/info/authors[author/text() = "E. F. Codd"]
return <Codd-paper>
  <title>{ $x/./title/text() }</title>
  <key>{ $x/./../@id}</key>
  { $x/./year }
</Codd-paper>
```

```
<Codd-paper>
  <title>A Relational Model of Data for Large Shared Data Banks.</title>
  <key>"5512431"</key>
  <year>1970</year>
</Codd-paper>
<Codd-paper>
  ...
</Codd-paper>
```

32

32

XQuery – example 3

```
let $allpapers := doc ("dblp.xml")/result/hits/
return <article-authors>
  <count> { fn:count($allpapers/hit) } </count>
{ for $paper in doc("dblp.xml")/result/hits/*/info
  let $pauth := $paper/authors
  return <paper> {$paper/title}
    <count> { fn:count($pauth/author) } </count>
  </paper>
} </article-authors>
```

```
<article-authors >
  <count>115</count>
  <paper>
    <title>A Relational Model of Data for Large Shared Data Banks.</title>
    <count>1</count>
  </paper>
  <paper>
    ...
  </article-authors >
```

33

33

XQuery - sorting

```
for $x in doc ("dblp.xml")/result/hits/*/info
  order by $x/title/text()
return $x
```

A co-relational model of data for large shared data banks.
 Fatal Flaws in SQL - Part ONE.
 Fatal Flaws in SQL - Part Two.

34

34

XQuery – querying & defining metadata

Can get a node's name by querying `name()`:

```
for $x in doc ("dblp.xml")/result/hits/hit/*
return name($x)
```

Can construct elements and attributes using **computed names**:

```
for $x in doc ("dblp.xml")/result/hits/hit/info*,
  $year in $x/year,
  $title in $x/title/text()
return
element { name($x) } {
  attribute { "year-" + $year } { $title }
}
```

35

35

XML in SQL Server

```
SELECT BusinessEntityID, FirstName,
       LastName, PersonType
FROM   Person.Person;
```

BusinessEntityID	FirstName	LastName	PersonType
1	Ken	Sánchez	EM
2	Terri	Duffy	EM

```
SELECT BusinessEntityID AS '@ID',
       FirstName AS "EmpName/First",
       LastName AS "EmpName/Last",
       PersonType AS "EmpType/Type"
FROM   Person.Person
FOR XML PATH('Person');
```

```
<Person ID="1">
  <EmpName>
    <First>Ken</First>
    <Last>Sánchez</Last>
  </EmpName>
  <EmpType>
    <Type>EM</Type>
  </EmpType>
</Person>
<Person ID="2">
  <EmpName>
    <First>Terri</First>
    <Last>Duffy</Last>
  </EmpName>
  <EmpType>
    <Type>EM</Type>
  </EmpType>
</Person>
```

...

36

XML in SQL Server

```
WITH XMLNAMESPACES(DEFAULT 'http://schemas.microsoft.com/sqlserver/2004/07/adventure-works/IndividualSurvey')
```

```
SELECT Demographics.value('/IndividualSurvey/Gender)[1]', 'nvarchar') AS text,  
       Demographics.query('/IndividualSurvey/Gender)[1]') AS XML,  
       Demographics.exist('/IndividualSurvey/Gender)[1]') AS Bool
```

```
FROM Person.Person;
```

text	XML	Bool
M	<p1:Gender xmlns:p1="http://schemas.microsoft.com/sqlserver/2004/07/adventure-works/IndividualSurvey">M</p1:Gender>	1
F	<p1:Gender xmlns:p1="http://schemas.microsoft.com/sqlserver/2004/07/adventure-works/IndividualSurvey">F</p1:Gender>	1
NULL		0

37

JSON - JavaScript Object Notation

- Lightweight data-interchange format
- It is easy for humans to read and write.
- It is easy for machines to parse and generate.
- It is based on a subset of the JavaScript Programming Language Standard ECMA-262 3rd Edition - December 1999.
- It is completely language independent but uses conventions that are familiar to programmers.

38

38

XML vs. JSON

```
<hit score="7" id="5512431">
  <info>
    <authors>
      <author>E. F. Codd</author>
      <author>Edgar F. Codd</author>
    </authors>
    <title>
      A Relational Model of Data for Large Shared Data Banks.
    </title>
    <venue>Commun. ACM</venue>
    <volume>13</volume>
    <number>6</number>
    <pages>377-387</pages>
    <year>1970</year>
    <type>Journal Articles</type>
    <key>journals/cacm/Codd70</key>
    <doi>10.1145/362384.362685</doi>
  </info>
</hit>
```

```
{
  "@score" : "7",
  "@id" : "5512431",
  "info" : {
    "authors" : [
      { "author" : { "text" : "E. F. Codd" } },
      { "author" : { "text" : "Edgar F. Codd" } },
    ],
    "title" : "A Relational Model of Data for Large...",
    "venue" : "Commun. ACM",
    "volume" : "13",
    "number" : "6",
    "pages" : "377-387",
    "year" : "1970",
    "type" : "Journal Articles",
    "key" : "journals/cacm/Codd70",
    "doi" : "10.1145/362384.362685"
  }
}
```

Object

Array

Value

39

39

JSON grammar

An **OBJECT** is an unordered set of name-value pairs.

An **OBJECT** begins with { and ends with }. Each name is followed by : and the name-value pairs are separated by ,.

An **ARRAY** is an ordered collection of values. An **array** begins with [and ends with]. Values are separated by ,.

A **VALUE** can be a **string** in double quotes, or a **number**, or **true** or **false** or **null**, or an **object** or an **array**. These structures can be nested.

40

40

JSON vs. XML

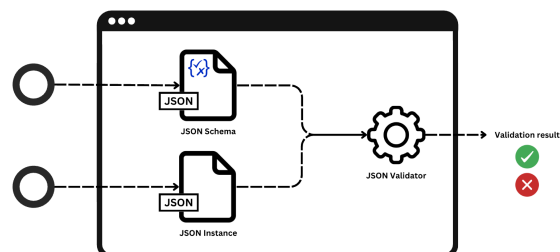
- JSON doesn't use end tag
- JSON is shorter
- JSON is quicker to read and write
- XML has to be parsed with an XML parser. JSON can be parsed by a standard JavaScript function.
- XML is much more difficult to parse than JSON
- For AJAX applications, JSON is faster and easier than XML

41

41

JSON Schema

- JSON Schema is a declarative language for annotating and validating JSON documents' structure, constraints, and data types.
- It helps you standardize and define expectations for JSON data.

source: <https://json-schema.org/>

42

42

Benefits of JSON Schema

- **Simplifying Testing and Validation:** JSON Schema reduces code complexity and development time by simplifying validation logic. It defines constraints for data structures, enabling the detection and prevention of errors, inconsistencies, and invalid data.
- **Facilitating Seamless Data Exchange:** JSON Schema establishes a common language for data exchange. It defines precise validation rules for data structures to create a shared understanding and increase interoperability across different systems and platforms.
- **Enhancing Data Documentation:** JSON Schema enables the creation of clear and standardized representations of data. This improves understanding and collaboration among developers, stakeholders, and collaborators, enhancing organizational efficiency.

source: <https://json-schema.org/>

43

43

JSON Schema

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://example.org/schemas/article.json",
  "title": "Publications",
  "type": "object",
  "required": ["@score", "@id", "info"],
  "properties": {
    "@score": {
      "type": "integer",
      "description": "Subsequent item"
    },
    "@id": {
      "type": "string",
      "pattern": "^([0-9]+)$",
      "description": "Unique identifier"
    },
    ...
  }
}
```

JSON

```
{
  "@score": "7",
  "@id": "5512431",
  "info": {
    "authors": {
      "author": {
        "text": "E. F. Codd"
      }
    },
    "title": "A Relational Model of Data for Large Shared Data Banks.",
    "venue": "Commun. ACM",
    "volume": "13",
    "number": "6",
    "pages": "377-387",
    "year": "1970",
    "type": "Journal Articles",
    "key": "journals/cacm/Codd70",
    "doi": "10.1145/362384.362685"
  }
}
```

44

44

JSON Schema

```
"author": {
  "type": "array",
  "items": {
    "type": "string"
  }
}
...
```

Multiple authors

```
"type": {
  "enum": ["Journal Articles", "Books", "PhD"]
},
```

JSON

```
"authors" : {
  "author" : {
    "text" : "E. F. Codd"
  }
},
...
```

```
"type" : "Journal Articles",
```

45

45

JSON in SQL Server

```
SELECT id,
  firstName AS "info.name",
  lastName AS "info.surname",
  age,
  dateOfBirth AS dob
FROM People
FOR JSON PATH;
```

```
[
  {
    "id": 2,
    "info": {
      "name": "John",
      "surname": "Smith"
    },
    "age": 25
  },
  {
    "id": 5,
    "info": {
      "name": "Jane",
      "surname": "Smith"
    },
    "dob": "2005-11-04T12:00:00"
  }
]
```

46

46

JSON in SQL Server

```

SELECT
    Name,
    Surname,
    JSON_VALUE(jsonCol, '$.info.address."Address Line 1"') AS Address,
    JSON_QUERY(jsonCol, '$.info.skills') AS Skills
FROM People
WHERE
    ISJSON(jsonCol) > 0 AND
    JSON_VALUE(jsonCol, '$.info.address.Town') = 'Belgrade' AND
    Status = 'Active'
ORDER BY
    JSON_VALUE(jsonCol, '$.info.address.PostCode');

```

47

47

Document databases

Relational

ID	first_name	last_name	cell	city	year_of_birth	location_x	location_y
1	'Mary'	'Jones'	'516-555-2048'	'Long Island'	1986	'-73.9876'	'40.7574'

ID	user_id	profession
10	1	'Developer'
11	1	'Engineer'

ID	user_id	name	version
20	1	'MyApp'	1.0.4
21	1	'DocFinder'	2.5.7

ID	user_id	make	year
30	1	'Bentley'	1973
31	1	'Rolls Royce'	1965

MongoDB

```

{
  first_name: "Mary",
  last_name: "Jones",
  cell: "516-555-2048",
  city: "Long Island",
  year_of_birth: 1986,
  location: {
    type: "Point",
    coordinates: [-73.9876, 40.7574]
  },
  profession: ["Developer", "Engineer"],
  apps: [
    { name: "MyApp",
      version: 1.0.4 },
    { name: "DocFinder",
      version: 2.5.7 }
  ],
  cars: [
    { make: "Bentley",
      year: 1973 },
    { make: "Rolls Royce",
      year: 1965 }
  ]
}

```

source: MongoDB.com

48

MongoDB

```

db.users.insertOne(
  {
    name: "sue",
    age: 26,
    status: "pending"
  }
)

```

← collection
 ← field: value
 ← field: value
 ← field: value } document

```

db.users.find(
  { age: { $gt: 18 } },
  { name: 1, address: 1 }
).limit(5)

```

← collection
 ← query criteria
 ← projection
 ← cursor modifier

```

db.users.updateMany(
  { age: { $lt: 18 } },
  { $set: { status: "reject" } }
)

```

← collection
 ← update filter
 ← update action

source: MongoDB.com

49

49

Application Programming Interfaces

API ARCHITECTURAL STYLES

	RPC	SOAP	REST	GraphQL
Organized in terms of	local procedure calling	enveloped message structure	compliance with six architectural constraints	schema & type system
Format	JSON, XML, Protobuf, Thrift, FlatBuffers	XML only	XML, JSON, HTML, plain text	JSON
Learning curve	Easy	Difficult	Easy	Medium
Community	Large	Small	Large	Growing
Use cases	Command and action-oriented APIs; internal high performance communication in massive micro-services systems	Payment gateways, identity management, CRM solutions, financial and telecommunication services, legacy system support	Public APIs, simple resource-driven apps	Mobile APIs, complex systems, micro-services



50

50

Thank you for your
attention!

51